

WEEK 1 CRASH PLAN: June 18-22 (5 Days)

June 16 (today) = done. June 17 = travel. Real work starts June 18 (Wednesday).

Two deadlines: Nimblize Report (Sun 22) + Trilogy DSA Test (Tue 23)

Trilogy Test Topics: Notes + Practice

The test covers: **Adhoc, Hashing, Maths, Bit Manipulation**

- Topic 1 = Easy level
 - Topic 2 = Medium level
-

TOPIC 1: ADHOC

What it means: Problems with no standard algorithm. You observe a pattern, use logic, and code it. Common in competitive programming.

Key skills needed:

- Careful reading of problem statement
- Identifying edge cases (empty input, single element, negative numbers)
- Simulation - doing exactly what the problem says step by step
- Pattern recognition from examples

Tricks to remember:

- If stuck, try brute force first, then optimize
- Always check: what happens with $n=0$, $n=1$, negative numbers?
- Look for mathematical patterns in the examples
- Sometimes sorting the input simplifies everything

Practice Problems (Easy):

#	Platform	Problem	Key Idea
1	LC #412	Fizz Buzz	Simple conditionals, modulo

2	LC #1295	Find Numbers with Even Number of Digits	Count digits
3	LC #1480	Running Sum of 1D Array	Prefix sum
4	LC #1672	Richest Customer Wealth	2D array row sum
5	LC #58	Length of Last Word	String traversal

Practice Problems (Medium):

#	Platform	Problem	Key Idea
6	LC #54	Spiral Matrix	Simulation with boundaries
7	LC #48	Rotate Image	Matrix rotation pattern
8	LC #73	Set Matrix Zeroes	In-place marking
9	LC #189	Rotate Array	Reverse trick
10	LC #238	Product of Array Except Self	Left-right product

TOPIC 2: HASHING

What it means: Using HashMap/HashSet to store and lookup data in $O(1)$ time. Most common data structure in interviews.

Key concepts:

```
// HashMap: stores key-value pairs
HashMap<String, Integer> map = new HashMap<>();
map.put("apple", 5);    // insert
map.get("apple");       // returns 5
map.containsKey("apple"); // returns true
map.getDefault("banana", 0); // returns 0 if not found

// HashSet: stores unique values only
HashSet<Integer> set = new HashSet<>();
set.add(5);             // insert
set.contains(5);        // returns true
set.size();             // number of elements
```

Common patterns:

1. **Frequency counting** - count occurrences of each element
2. **Two Sum pattern** - store complement, check if it exists
3. **Duplicate detection** - add to set, check if already present
4. **Grouping** - group elements by some key (anagrams, etc.)

Practice Problems (Easy):

#	Platform	Problem	Key Idea
1	LC #1	Two Sum	Store complement in map
2	LC #217	Contains Duplicate	HashSet for duplicates
3	LC #242	Valid Anagram	Frequency count comparison
4	LC #383	Ransom Note	Character frequency map
5	LC #349	Intersection of Two Arrays	HashSet intersection

Practice Problems (Medium):

#	Platform	Problem	Key Idea
6	LC #49	Group Anagrams	Sort as key, group in map
7	LC #347	Top K Frequent Elements	Frequency map + sorting
8	LC #128	Longest Consecutive Sequence	HashSet + expand
9	LC #3	Longest Substring No Repeating	Sliding window + set
10	LC #560	Subarray Sum Equals K	Prefix sum + hashmap

TOPIC 3: MATHS

What it means: Problems involving mathematical formulas, number properties, modular arithmetic.

Key concepts to know:

GCD (Greatest Common Divisor):

- Euclidean algorithm: $\text{gcd}(a, b) = \text{gcd}(b, a \% b)$, base: $\text{gcd}(a, 0) = a$

LCM (Least Common Multiple):

- $\text{lcm}(a, b) = (a * b) / \text{gcd}(a, b)$

Prime numbers:

- Check divisibility up to $\sqrt{n}$ only
- Sieve of Eratosthenes for generating all primes up to N

Modular arithmetic:

- $(a + b) \% m = ((a \% m) + (b \% m)) \% m$
- $(a * b) \% m = ((a \% m) * (b \% m)) \% m$

Power of 2:

- $n > 0 \ \&\& \ (n \& (n-1)) == 0$

Factorial / Combinatorics:

- $nCr = n! / (r! * (n-r)!)$

Java helper code:

```
// GCD
int gcd(int a, int b) {
    while (b != 0) {
        int temp = b;
        b = a % b;
        a = temp;
    }
    return a;
}

// Check prime
boolean isPrime(int n) {
    if (n <= 1) return false;
    for (int i = 2; i * i <= n; i++) {
        if (n % i == 0) return false;
    }
    return true;
}
```

Practice Problems (Easy):

#	Platform	Problem	Key Idea
1	LC #9	Palindrome Number	Reverse half the number
2	LC #1979	Find GCD of Array	Euclidean GCD
3	LC #204	Count Primes	Sieve of Eratosthenes
4	LC #326	Power of Three	Repeated division
5	LC #7	Reverse Integer	Modulo + overflow check

Practice Problems (Medium):

#	Platform	Problem	Key Idea
6	LC #50	Pow(x, n)	Fast exponentiation
7	LC #2	Add Two Numbers	Carry arithmetic on linked list
8	LC #29	Divide Two Integers	Bit shifting division
9	LC #43	Multiply Strings	Grade school multiplication
10	LC #1492	The kth Factor of n	Factor enumeration

TOPIC 4: BIT MANIPULATION

What it means: Using binary operations (AND, OR, XOR, shifts) to solve problems efficiently.

Key operations to memorize:

AND (&): $1 \& 1 = 1$, everything else = 0

Use: Check if bit is set, clear bits

OR (|): $0 \mid 0 = 0$, everything else = 1

Use: Set a specific bit

XOR (^): Same bits = 0, different = 1

Use: Find unique element, toggle bits

KEY PROPERTY: $a \wedge a = 0$, $a \wedge 0 = a$

LEFT SHIFT (<<): Multiply by 2

$5 \ll 1 = 10, 5 \ll 2 = 20$

RIGHT SHIFT ($\gg$): Divide by 2

$10 \gg 1 = 5$

TRICKS:

Check if i th bit is set: $(n \gg i) \& 1$

Set i th bit: $n \mid (1 \ll i)$

Clear i th bit: $n \& \sim(1 \ll i)$

Toggle i th bit: $n \wedge (1 \ll i)$

Check power of 2: $n > 0 \&\& (n \& (n-1)) == 0$

Count set bits: `Integer.bitCount(n)` in Java

Get lowest set bit: $n \& (-n)$

The most important trick - XOR for finding single element:

```
// If every element appears twice except one, XOR all = answer
int single = 0;
for (int num : nums) {
    single ^= num; // pairs cancel out (a ^ a = 0)
}
return single;
```

Practice Problems (Easy):

#	Platform	Problem	Key Idea
1	LC #136	Single Number	XOR all elements
2	LC #191	Number of 1 Bits	Count set bits
3	LC #231	Power of Two	$n \& (n-1) == 0$
4	LC #338	Counting Bits	DP + bit pattern
5	LC #190	Reverse Bits	Shift and build

Practice Problems (Medium):

#	Platform	Problem	Key Idea
6	LC #137	Single Number II	Bit counting mod 3

7	LC #260	Single Number III	XOR + grouping by bit
8	LC #78	Subsets	Bitmask enumeration
9	LC #371	Sum of Two Integers	Add using XOR + AND
10	LC #201	Bitwise AND of Range	Common prefix

5-DAY SCHEDULE (June 18-22)

Strategy: Mornings = Trilogy prep, Afternoons = Nimblize report

Day	Morning (2.5 hrs) - Trilogy Prep	Afternoon (2 hrs) - Nimblize Report	Evening (30 min)
Wed 18	HASHING: Read notes above + solve LC #1, #217, #242, #349 (Easy)	Nimblize: Research APIs (REST, HTTP methods, JSON) + take notes	Zift daily challenge
Thu 19	HASHING Medium + ADHOC Easy: Solve LC #49, #347, #412, #1480	Nimblize: Database Design section + draw ER diagram (use draw.io)	Zift daily challenge
Fri 20	MATHS: Read notes + solve LC #9, #1979, #204, #7 + try LC #50	Nimblize: Architecture diagram + Cloud Fundamentals section	Zift daily challenge
Sat 21	BITS: Read notes + solve LC #136, #191, #231, #338 + try LC #137	Nimblize: Infrastructure Design + Deployment diagram	Zift daily challenge
Sun 22	MIXED REVISION: 1 problem from each topic (timed, 15 min each). Try: LC #128, #238, #29, #260	Nimblize: Compile, polish, SUBMIT by 7 PM	Zift daily challenge

If you can only do 2 hours total on any day:

Priority order: Trilogy prep > Nimblize report (test is harder to recover from than a late report)

Night Before Test (Mon 22 night):

- Re-read the notes above (30 min)
- Review your solved problems - don't solve new ones
- Sleep properly

MINIMUM SURVIVAL CHECKLIST FOR TRILOGY TEST

If you're short on time, AT MINIMUM solve these 8 problems:

#	LC ID	Problem	Topic	Why
1	#1	Two Sum	Hashing	Asked EVERYWHERE
2	#217	Contains Duplicate	Hashing	Basic HashSet
3	#136	Single Number	Bits	Classic XOR
4	#9	Palindrome Number	Maths	Basic number manipulation
5	#1480	Running Sum	Adhoc	Prefix sum pattern
6	#49	Group Anagrams	Hashing (Med)	HashMap grouping
7	#191	Number of 1 Bits	Bits	Bit counting
8	#204	Count Primes	Maths	Sieve - commonly asked

UPDATED FULL TIMELINE SUMMARY

Period	Focus	Deadlines
June 16-17	Lost (today + travel)	-
June 18-22	Trilogy prep + Nimblize report	Nimblize: Sun 22 / Trilogy: Tue 23
June 23-July 6	DBMS learning + TransitPulse DB + Java + DSA	TransitPulse checkpoint
July 7-10	TransitPulse DB completion + Spring Boot start	TransitPulse: July 10
July 11-20	Spring Boot + DAA basics + revision	Semester prep

NOTE: College daily DSA challenges are on the **Zift platform** (not LeetCode). Do those first each evening. The LeetCode problems listed above are specifically for Trilogy test prep - do them in the morning block.